

SSTI (Server-Side Template Injection)

SSTI is arguably the highest-ceiling vulnerability class you've studied in this web security arc — where SQLi gives an attacker control over a database query and XXE gives control over an XML parser, SSTI can hand an attacker **direct code execution on the server itself**. It was first formally documented by PortSwigger Research in a 2015 paper explicitly framed around this exact idea: template injection as a path to full RCE on modern web apps.

1. What SSTI Actually Is

SSTI happens when an attacker can inject **native template syntax** into a template, which the server-side template engine then evaluates as code rather than as inert data. Template engines exist to combine a fixed template with dynamic data and produce HTML — the vulnerability arises specifically when **user input becomes part of the template itself**, instead of being passed into the template as a data value.

```
Template engines = fixed template + dynamic DATA → rendered HTML
SSTI            = fixed template + dynamic user-controlled TEMPLATE SYNTAX → arbitrary code evaluated
```

2. The Critical Distinction — Data vs. Template

This is the entire vulnerability in one comparison.

Safe (data passed in properly):

```
$output = $twig->render("Dear {first_name},", array("first_name" => $user.first_name));
```

The user's name is passed as **data** to fill a placeholder — the template structure itself is fixed and never touched by user input, so there's nothing to inject into.

Vulnerable (user input concatenated into the template string itself):

```
$output = $twig->render("Dear " . $_GET['name']);
```

Here, part of the **template string itself** is built dynamically from a `GET` parameter. Since template syntax is evaluated server-side, an attacker can place actual template directives inside that parameter:

```
http://vulnerable-website.com/?name={{bad-stuff-here}}
```

The template engine will attempt to *evaluate* `{{bad-stuff-here}}` as template syntax, not just print it as text.

Direct parallel to your SQLi guide: this is precisely the same root-cause pattern as building a SQL query through string concatenation instead of parameterization — user input is meant to be *data*, but ends up embedded in the *structure* the engine interprets as instructions.

3. How This Happens in Practice

Accidental (poor template design): developers concatenate several string fragments, some user-controlled, into a single template before rendering — often by people unfamiliar with the security implications of what "the template" actually means to the engine. This is functionally the templating-engine equivalent of a poorly written SQL prepared statement.

Intentional (by design, and often riskier): some applications deliberately let privileged users — content editors, marketers, admins — submit or edit custom templates as a feature. If an attacker manages to compromise one of those privileged accounts, they inherit direct template-injection capability by design, not by accident — a good reminder that "authorized" functionality can still be a severe risk if the authorized user's account is compromised.

4. Impact — Why SSTI Is So Dangerous

The severity range is genuinely wide, and depends heavily on the specific template engine and how the application uses it:

- **Worst case: Remote Code Execution (RCE)** — full control of the backend server, which can then become a pivot point into internal infrastructure

(exactly the kind of foothold your Netcat reverse-shell guide covers catching)

- **Even without full RCE:** SSTI often still enables reading sensitive files/data from the server, similar in outcome to an XXE file-read primitive
- **In rare cases:** some SSTI instances genuinely pose little real risk, depending on what capabilities the specific engine/sandbox exposes

The RCE ceiling is what sets SSTI apart from most other injection classes you've studied — SQLi's worst case is usually data compromise (unless the DB config allows OS command execution); SSTI's worst case is routinely full server takeover, directly.

5. The SSTI Methodology — Detect → Identify → Exploit

This three-step process is the standard structured approach, and mirrors the "fingerprint before you exploit" discipline from your SQLi guide's database version-detection step.

Step 1: Detect

Fuzz with special characters:

```
${{<[%'"]}}%\
```

Injecting this sequence of characters commonly used across various template syntaxes and watching for an exception/error is a fast first signal — an error suggests the input is being *interpreted*, not just displayed as text.

Beyond fuzzing, SSTI shows up in two distinct contexts, each needing its own test:

a) Plaintext context — mistaken for XSS

```
render('Hello ' + username)
```

Testing:

```
http://vulnerable-website.com/?username=${7*7}
```

If the response shows `Hello 49` instead of `Hello  $\$ \{7*7\}$` , the math was evaluated server-side — proof the input is reaching the template engine, not just being echoed as HTML. This is genuinely easy to misdiagnose as plain XSS if you stop at the first HTML-injection test and never try a mathematical expression.

b) Code context — easily missed entirely

```
greeting = getQueryParameter('greeting')
engine.render("Hello {"+greeting+"}", data)
```

```
http://vulnerable-website.com/?greeting=data.username
→ renders: Hello Carlos
```

This context is genuinely sneaky — it looks and behaves almost exactly like a simple hashmap/variable lookup, with no obvious XSS signal at all. Testing methodology:

```
Step 1: http://vulnerable-website.com/?greeting=data.username<tag>
        → if XSS-safe, this comes back blank, encoded, or errors – NOT injected raw

Step 2: http://vulnerable-website.com/?greeting=data.username}}<tag>
        → attempting to break OUT of the template expression using common closing syntax, then inject HTML immediately after

If step 2 renders: Hello Carlos<tag>    (with the tag actually interpreted)
→ confirmed SSTI: you successfully closed the template expression early and injected content that gets rendered afterward
```

Step 2: Identify the Template Engine

Once you've confirmed injection is happening, you need to know **which** engine you're dealing with, since exploitation syntax is engine-specific — directly parallel to fingerprinting the database engine in your SQLi guide before writing platform-specific payloads.

Error messages often just tell you outright:

```
<%=foobar%>    (invalid Ruby ERB syntax)

→ triggers:
(erb):1:in `<main>': undefined local variable or method `foobar' for main:Object (Name Error)
```

A verbose error message like this identifies both the engine (ERB) and sometimes the version directly — a huge shortcut when available.

When errors aren't verbose enough, use differential testing — submit engine-specific syntax and observe which ones are actually evaluated vs. which throw errors or render literally, narrowing down the candidates by elimination.

Important trap to know about: the exact same payload can succeed in *multiple* different engines with *different* results:

```
{{7*'7'}}  
→ Twig: 49 (treats '7' as a number, multiplies normally)  
→ Jinja2: 777777 (treats '7' as a string, repeats it 7 times)
```

A single successful-looking response is not enough to conclusively identify the engine — cross-check with a second, differentiating payload before committing to an exploitation strategy built around a specific engine's syntax.

Step 3: Exploit

Once the engine is identified, exploitation typically follows one of these paths:

- **Read the engine's own documentation** for built-in functions/objects that expose dangerous capability (file access, OS command execution, subprocess spawning)
- **Explore developer-supplied objects** — many frameworks expose specific application objects into the template context (a `config` object, a `request` object) that can sometimes be walked/chained to reach dangerous underlying functionality
- **Construct a custom object-chain exploit** — starting from any object reachable in the template context and pivoting through its accessible attributes/methods until you reach something that executes code (this is exactly the technique behind well-known Jinja2 SSTI-to-RCE chains, which walk Python's object model from a seemingly harmless string object all the way to `os.popen`)

6. Example — A Complete Walkthrough (Jinja2/Flask, Conceptual)

1. Detect:

```
?name={{7*7}}
```

→ response shows "49" instead of "{{7*7}}" → confirmed injection

2. Identify:

```
?name={{7*'7'}}
```

→ response shows "7777777" → this behavior is specific to Jinja2/Python

(Twig/PHP would show "49" instead, per Section 5's differential example)

3. Exploit (conceptual object-chain toward RCE):

```
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
```

→ walks from a built-in template object, through Python's introspection

capabilities, to the os module, achieving command execution

This chain looks intimidating at first, but it's fundamentally the same idea as chaining an IDOR into a password reset flow from your IDOR guide, or chaining a blind XXE through an external DTD — a sequence of individually-understandable steps that compound into something far more severe than any single step alone.

7. SSTI vs. Client-Side Template Injection — Worth the Distinction

Client-side template injection happens in JavaScript-based templating (AngularJS-style expressions evaluated in the browser) — it's essentially a variant of XSS, executing in the victim's browser, limited to what browser-side JavaScript can do.

Server-side template injection executes on the server itself, which is exactly why it's categorized as potentially far more dangerous — full server compromise vs. a single victim's browser session. Don't conflate the two just because the syntax often looks similar (`{{...}}` appears in both contexts).

8. Prevention — How to Actually Avoid This

The strongest fix: never let users modify or submit templates at all. Where this is genuinely unavoidable due to business requirements (the "privileged content editor" scenario from Section 3):

a) Use a "logic-less" template engine (like Mustache) wherever possible — logic-less engines are specifically designed to only support variable substitution, with no capability for evaluating expressions, walking object chains, or calling functions. This structurally removes the entire attack surface rather than trying to filter dangerous input.

b) Sandbox execution — run user-supplied template code in an environment with dangerous modules/functions stripped out entirely. Worth knowing this is **inherently difficult and prone to bypass** — sandbox escape research is its own entire subfield, and template engine sandboxes have a long history of being circumvented through creative object-chaining exactly like Section 6's example.

c) Defense-in-depth via infrastructure isolation — accept that arbitrary code execution may eventually be achievable despite best efforts, and deploy the templating environment inside a locked-down container (Docker) so that even a successful SSTI-to-RCE chain is contained rather than reaching the full production server/network.

The core architectural principle, same as SQLi's parameterized-query lesson: keep user input strictly as **data**, never allow it to become part of the **template/query structure itself** — every defense above is really just a different way of enforcing that same boundary.

9. Practical Lab Example

WebGoat and PortSwigger's own Web Security Academy labs both include dedicated SSTI modules — since you've referenced PortSwigger directly for this topic, working through their SSTI lab set (linked from this same page) is the most direct hands-on path.

Practice flow:

1. Find a parameter that renders back into the page in an unusual way (not obviously reflected exactly as submitted)
2. Try the `{{7*7}}` math test first – fastest, clearest signal
3. If nothing, try breaking out of a code-context expression with `}}`
4. Once confirmed, fingerprint the engine via error messages or differential payloads (`{{7*'7'}}`)
5. Consult that specific engine's known SSTI-to-RCE technique (search "[engine name] SSTI exploit" - e.g., "Jinja2 SSTI to RCE")

10. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-1336 (Improper Neutralization of Special Elements Used in a Template Engine) — a child of the broader CWE-74 Injection family from your CWE guide, same lineage as CWE-89 (SQLi)
OWASP Top 10:2025	A05:2025 Injection
CVSS	Frequently scores Critical when RCE is achievable — AV:N/AC:L/PR:N/UI:N/C:H/I:H/A:H is common for unauthenticated SSTI-to-RCE
Cyber Kill Chain	Exploitation → Installation (a successful RCE chain often directly enables dropping a webshell or catching a reverse shell)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application), and once RCE is achieved, whatever post-exploitation techniques follow (T1059 Command and Scripting Interpreter)

Quick Reference Cheat Sheet

Detection:

`{{{<%['"]}}}%\` ← fuzz string, look for errors

Plaintext context:

`?param={{7*7}}` ← if output shows 49, confirmed

Code context:

`?param=data.username<tag>` ← test 1: is this just XSS-safe?

`?param=data.username}}<tag>` ← test 2: break out of expression

Engine fingerprinting:

`{{7*'7'}}` → 49 (Twig) vs 7777777 (Jinja2)

`<%=foobar%>` → verbose Ruby ERB error reveals engine directly

Prevention:

- Never let user input become part of the template structure
 - Use logic-less engines (Mustache) where possible
 - Sandbox + container isolation as defense-in-depth
-